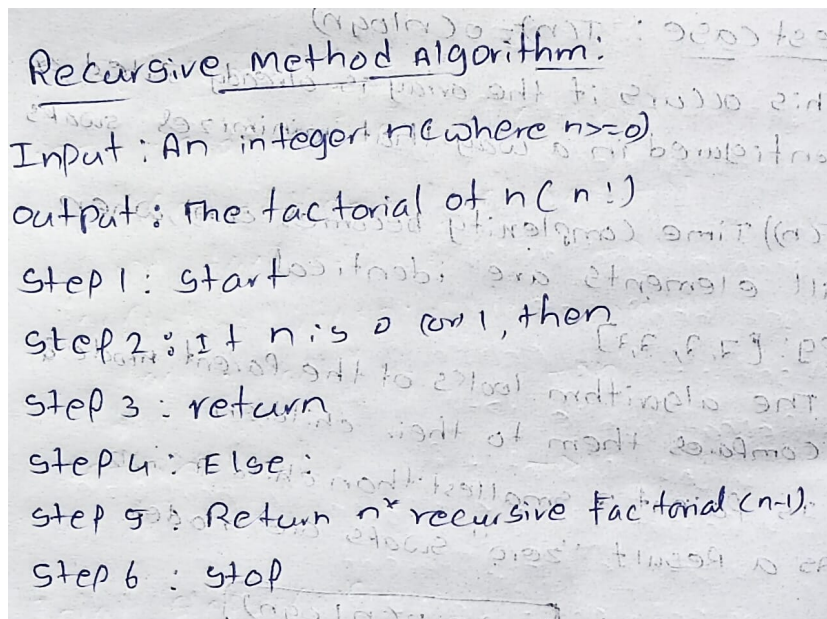


PRACTICAL-4

Aim : To implement and analyze the execution time of the factorial algorithm using both iterative and recursive approaches

1) Recursive Method :



Recursive method algorithm:
Input : An integer n where $n \geq 0$
Output : The factorial of n ($n!$)
Step 1 : start
Step 2 : If n is 0 or 1, then
Step 3 : return
Step 4 : Else :
Step 5 : Return $n \times$ recursive factorial ($n-1$).
Step 6 : stop

Program / Code :

```
#include <iostream>
using namespace std;
```

```
class Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
        else
        {
            int result = n * factr(n - 1);
            return result;
        }
    }
};
```

```
}  
};  
  
int main()  
{  
    int n, result;  
  
    Factorial obj;  
  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
  
    result = obj.factr(n);  
  
    cout << "Factorial of " << n << " is " << result << endl;  
  
    return 0;  
}
```

Output :

```
Enter the value to find its factorial: 5  
Factorial of 5 is 120
```

Time Complexity :

Best Case : $O(n)$

Average Case : $O(n)$

Worst Case : $O(n)$

Time complexity:-

```

int recursive Factorial (int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return n * recursive Factorial (n-1);
}

```

Time to calculate factorial of size n:

$$T(n) = T(n-1) + O(1)$$

$$T(n) = T(n-1) + 1$$

By substitution / Iterative method

$$T(n) = T(n-1) + 1$$

$$T(n) = [T(n-2) + 1] + 1 \rightarrow T(n-2) + 2$$

$$T(n) = [T(n-3) + 1] + 2 \rightarrow T(n-3) + 3$$

$$T(n) = T(n-k) + k$$

let $n-k=1 \Rightarrow k=n-1$

Substitute k value back into equation

$$T(n) = T(1) + (n-1)$$

Base condition: $T(1) = 1$

$$T(n) = 1 + n - 1$$

$$T(n) = n$$

$\therefore$ we take highest order term

$$T(n) = O(n)$$

* Best case: $O(n)$
 * Average case: $O(n)$
 * Worst case: $O(n)$

2) Iterative Method :

Algorithm :

Factorial using Iterative Method

Input : An integer n (where $n \geq 0$)
output : The factorial of n ($n!$)

step 1 : start
step 2 : Initialize fact = 1
step 3 : For $i = 1$ to n do:
step 4 : $\text{fact} = \text{fact} * i$
step 5 : End for
step 6 : Return Fact
step 7 : stop

Program / Code :

```
#include <iostream>
using namespace std;
```

```
class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;

        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }

        return result;
    }
};
```

```
int main()
```

```
{  
  
    int n, result;  
  
    Factorial obj;  
  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
  
    result = obj.fact(n);  
  
    cout << "Factorial of " << n << " is " << result << endl;  
  
    return 0;  
}
```

Output :

```
Enter the value to find its factorial: 10  
Factorial of 10 is 3628800
```

Time complexity :

Best Case : $O(n)$

Average Case : $O(n)$

Worst Case : $O(n)$

Time complexity

```

int iterative Factorial (int n) {
    if (n == 0 || n == 1) return 1;
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}
    
```

Total steps $O(n)$:

$$T(n) = 1 + 1 + (n+1) + n + 1$$

$$= 2n + 4$$

$\therefore$ we take highest order term

$$T(n) = O(n)$$

- * Best case : $O(n)$
- * Average case : $O(n)$
- * Worst case : $O(n)$

Conclusion : The iterative method is faster and more reliable because it uses a direct loop. The recursive method is slower because managing multiple function calls takes more time.